



Claude Code Training

A deep dive into the tool, the context, and the compounding advantage — built for your role, your work, your context.

CLAUDE TRAINING SESSION

Session Map

This training is structured as a progressive journey from understanding the ecosystem to building role-specific artefacts that compound in value every day.

01	02
Part 0 — The Claude Ecosystem Four products, one AI, very different jobs.	Parts 1–1.5 — What Is Claude Code + Co-work Agentic CLI, multi-step operators, and the practical difference.
03	04
Parts 2–6 — Foundation Organisational benefit, safety rules, CLEAR prompting, context files, and the four phases.	Parts 7–12 — Role Tracks, Capstone & Closing Role-specific work, compound effect, artifact map, capstone project, and next steps.

❑ **Role-specific track (Part 7):** Find your role section and work through it completely. In a classroom, role groups work simultaneously. In a webinar or self-paced read, go directly to your role and skip the others.

The Tool That Changes Every Week

Every time you read about Claude Code, it seems different from the last time. New features. New terminology. New capabilities. Tools that didn't exist last month. **This is not a bug.** It is the fastest-moving capability in enterprise software right now.

The Wrong Responses

- **Freeze** — waiting until things "settle down" before learning
- **Thrash** — chasing every update, never building anything stable

This Training's Approach

We teach you the underlying model the part that does not change so that when new features arrive, you know exactly where they fit.

- Claude is only as useful as the context you give it
- That context can be made persistent, reusable, and role-specific
- Building context is a one-time investment that compounds daily

Part 0

The Claude Ecosystem

Four Products. One AI. Very Different Jobs.

Before learning Claude Code, you need to know where it sits. Anthropic makes four distinct products. Most people discover them out of order and spend months confused about which one does what.

Product	What it is	Who uses it	Where
Claude.ai	Conversational AI in a browser or app	Everyone — the starting point	claude.ai
Projects	Persistent workspaces inside Claude.ai	Individuals and small teams	Inside claude.ai
Claude Co-work	Claude as an operator — takes actions, browses, runs multi-step tasks	Power users, operators	claude.ai / beta
Claude Code	Agentic CLI — works inside your codebase and project files	Developers and technical roles	Terminal / VS Code

📌 These are not versions of each other. They are different tools for different jobs. This training is primarily about Claude Code — but you will use all four.

Claude.ai — The Starting Point

Claude.ai is the browser and mobile interface most people encounter first. It is a conversational tool: you type, Claude responds. Each conversation is independent unless you use Projects.

Custom Instructions

Tell Claude how to behave across all your conversations. Your default tone, role, preferred format. Set once, applied always — the claude.ai equivalent of CLAUDE.md.

Artifacts

Substantial output — documents, code, tables, HTML — placed in a separate panel. Iterate without cluttering the chat. Separates *output* from *conversation*.

Projects

Persistent workspace with continuous memory of documents, instructions, and history. The closest claude.ai gets to Claude Code's context model — without file system access.

Connectors

Connect to Gmail, Google Drive, Slack, Jira, Confluence, Notion. Read emails, search Drive, summarise Slack threads — without leaving chat. A simpler, UI-driven integration layer.

Projects vs CLAUDE.md

Both give Claude persistent context — but at very different levels of depth and capability. Understanding the distinction helps you choose the right tool for each situation.

Projects (Claude.ai)

- UI-based — no files needed
- Upload documents as context
- No file system access
- No agentic actions
- Good for any role
- Shared team projects available

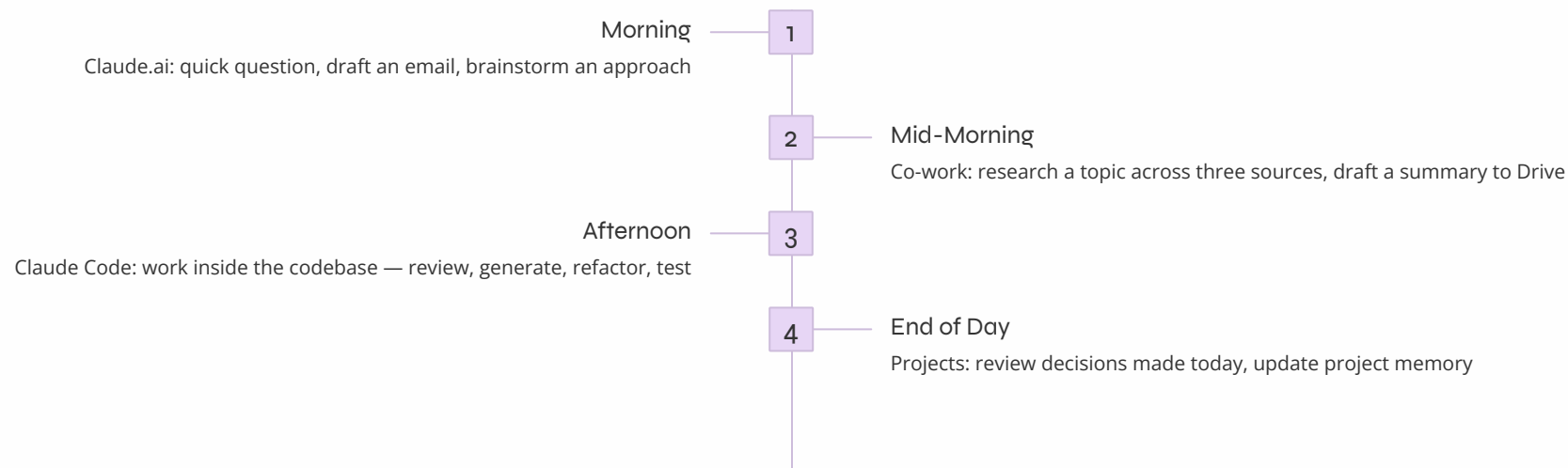
CLAUDE.md (Claude Code)

- File-based — lives in your project folder
- Reads your actual project files directly
- Full file system access
- Can create, edit, run things
- Most powerful for technical roles
- Version-controlled with your codebase

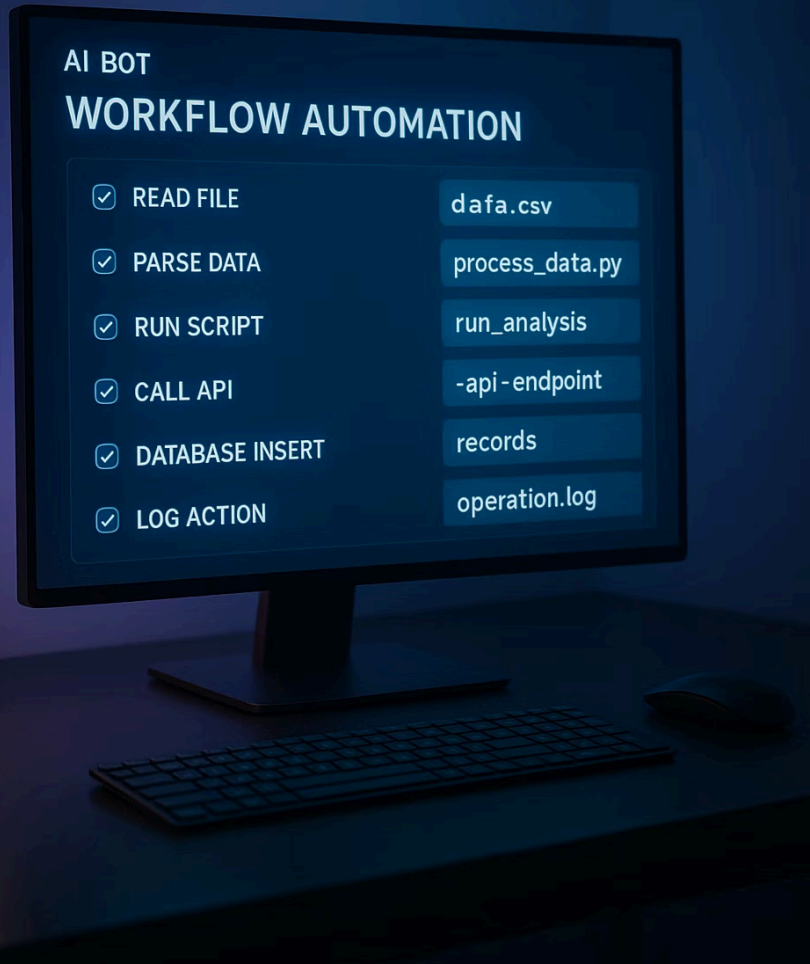
When you want Claude to know your documents without pasting them every time, or when your team needs a shared starting point with consistent context — Projects are the right tool. When you need depth, actions, and file access — Claude Code is the answer.

How the Four Products Work Together

They are not competing. They complement each other. A real workday might look like this:



□ The context files you build in Claude Code — CLAUDE.md, memory.md, SKILL.md — are the deepest form of persistent context. Project instructions in Claude.ai are the lighter-weight version. They serve the same purpose at different levels of depth.



Part 1

Not a Chatbot. Not a Copilot. Something New.

Most people assume Claude Code is smarter autocomplete. A fancier Copilot that lives in the editor and suggests the next line. It is not that. Claude Code is a **command-line agent** that runs in your terminal, reads your project files, understands the structure of your work, and **takes actions** inside your project.

Claude Code Is Different in Three Fundamental Ways



It Reads Your Project

Before you type a single word, Claude Code has already read your files, your folder structure, your code, and a file called CLAUDE.md that you write and maintain. It walks in already briefed.



It Acts

Claude Code does not just suggest. It creates files. It edits code. It runs commands. It executes tests. It does the work — within the boundaries you define and with your explicit approval at each significant step.



It Remembers Your Rules

Every session begins with Claude reading CLAUDE.md. Whatever you have written there — your constraints, your conventions, your context, your preferences — is applied automatically, every time, without repeating yourself.

Claude.ai vs Claude Code — The Practical Difference

Dimension	Claude.ai (browser)	Claude Code (terminal)
Memory	Starts fresh every session	Reads CLAUDE.md automatically every session
File access	None — you paste content in	Reads your project files directly
Actions	Suggests text	Creates, edits, runs things in your project
Workflow	Copy-paste	Integrated — works in your environment
Context	You rebuild it every time	Persists across sessions via context files

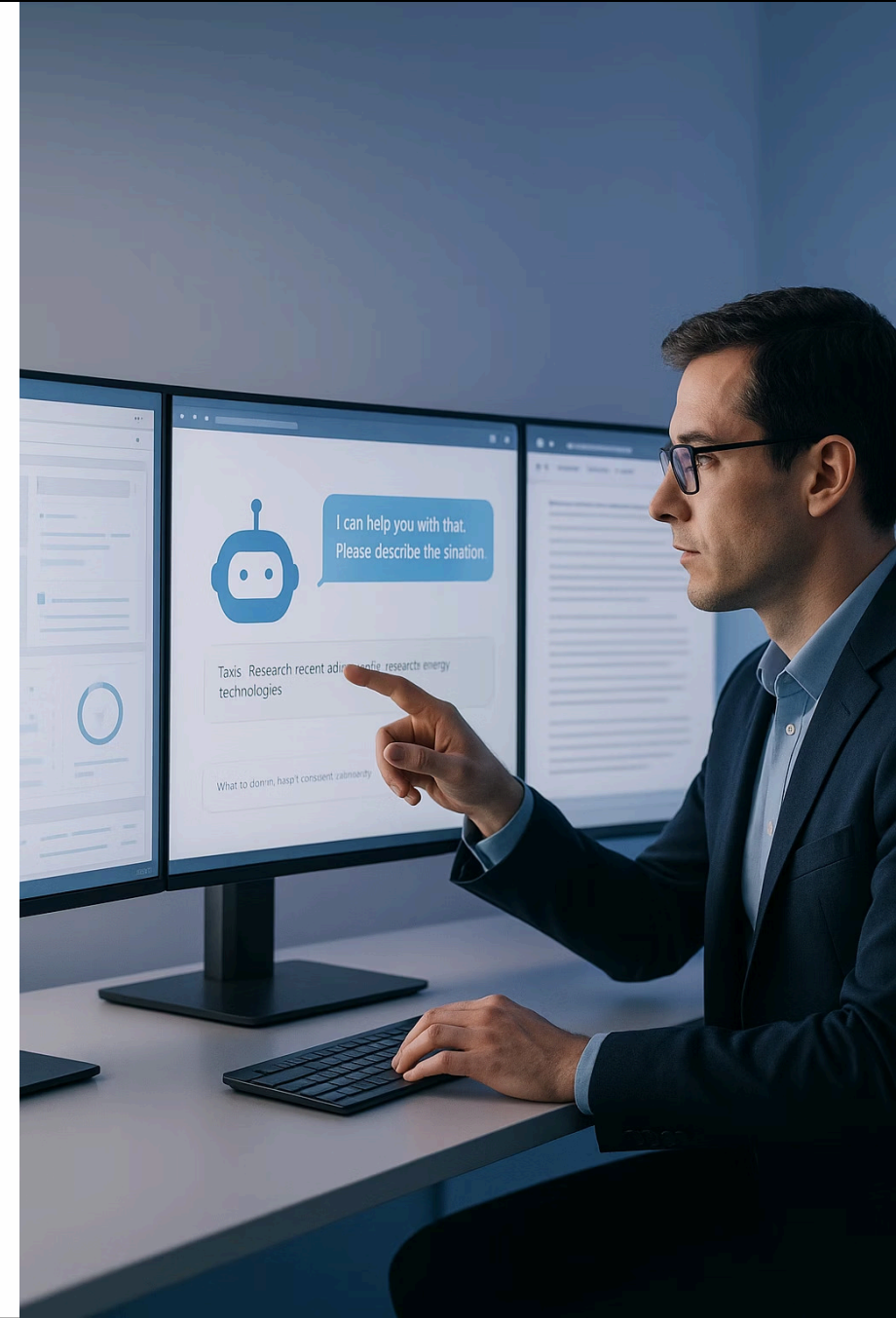
❏ **What Claude Code is NOT:** It is not infallible. It is not autonomous. It is not a replacement for your judgment. It is not a magic context reader. Every significant action requires your approval. The human is always in the loop.

Part 1.5

Claude Co-work: Claude as an Operator

Most Claude usage is single-turn: you ask, Claude answers. **Co-work breaks that pattern.** Co-work is Claude operating as an agent — taking a goal and executing a sequence of steps to reach it, using connected tools, browsing the web, and working across apps without you manually bridging each step.

This is not automation in the engineering sense. It does not require code. It is closer to **delegation**: you describe what you want done, Co-work figures out the steps, executes them, and returns the result.



What Co-work Can Do



Research Workflows

Give Co-work a question or topic. It browses the web, reads multiple sources, synthesises the findings, and returns a structured summary. What would take 45 minutes of tabbing between browser windows takes 3 minutes of reviewing Claude's output.



Multi-Step Task Execution

Co-work can chain tasks that would normally require you to switch between tools. Read from one place, process, write to another. Check Gmail, summarise action items, draft replies — in one workflow.



Reusable Skills

Workflows you use repeatedly can be saved as Skills in Co-work — invoked by name rather than re-described each time. The Co-work equivalent of SKILL.md in Claude Code. Example: "Weekly digest" — pull last week's Jira updates, summarise by project, draft standup notes.

Co-work and Claude Code — Complementary, Not Competing

A common question: if Co-work can do multi-step workflows, why use Claude Code at all? The answer is what each one can access.

Claude Co-work

- Works with the web and connected cloud apps
- Gmail, Drive, Slack, Jira, Confluence
- Information workflows — reading, synthesising, writing
- No local file system access
- Cannot run commands or execute code

Claude Code

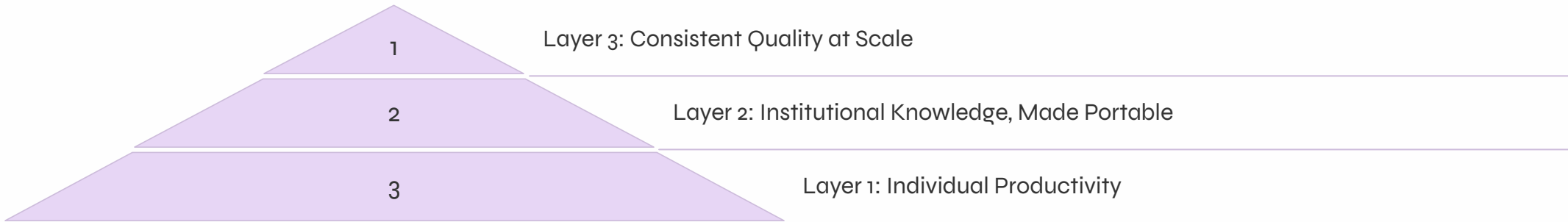
- Works inside your project — files, code, commands, tests, builds
- Full file system access
- Runs, edits, and creates at a deep technical level
- Cannot access the web or cloud apps directly
- Most powerful for technical roles

📌 For most people in technical roles, the answer is **both**: Co-work for information and communication workflows, Claude Code for everything inside the codebase and project.

Part 2

How It Helps Organisations

The Three Layers of Benefit



Layer 1 is real and immediate. Layers 2 and 3 are where the transformational benefit lives — and they are only achievable when teams invest in building their context files together.

Layer 1: Individual Productivity

The immediate, visible benefit. A developer produces more in a day. A PM reclaims hours every week. An ops engineer does in 30 minutes what previously took 2 hours. Real — but the least interesting layer.

Layer 2: Institutional Knowledge, Made Portable

Every organisation has knowledge that lives in senior people's heads. CLAUDE.md, memory.md, and SKILL.md are a **forcing function for making implicit knowledge explicit**. When a team builds these files together, they are documenting what they know — and that documentation becomes institutional property.

Layer 3: Consistent Quality at Scale

When a team shares a CLAUDE.md, every member works from the same rules, the same conventions, the same quality standards. A new team member who joins with an established CLAUDE.md has access to months of accumulated context from their first day.



Part 3

Foundation: Three Questions That Must Be Answered First

Before any role-specific work, every person must answer three questions. These answers become Section 1 of their CLAUDE.md. They are not suggestions. They are preconditions for responsible use of Claude Code in any real work environment.

Question 1: What Data Must Never Enter a Prompt?

Claude Code sends prompts to a language model. That model processes whatever you include. Your organisation almost certainly has policies about what data can enter an external system. If you are not sure — find out before doing any real work.

Role	Data that must never go in
Developer	Production credentials, API keys, database connection strings, live customer records, PII from production systems
Data & Analytics	Raw customer tables with identifiable fields, unreleased financial results, PII columns, model training data with personal data
QA & Testing	Production database records, real payment details, live customer credentials, actual user PII in bug reports
PM / Product	Individual performance data, salary information, confidential board decisions, client NDA-protected financials
Business Analyst	Client-confidential process details, competitor intelligence under NDA, legally sensitive information
Ops & Support	Production passwords and keys, customer data in logs, security scan results, network topology, access controls

📌 **The rule:** Before you paste anything into Claude Code — ask: *"Would I be comfortable if this appeared in a system I do not control?"* If not, anonymise it, summarise it, or leave it out.

Question 2: What Must Claude Never Do Without Explicit Approval?

Claude Code is an agent. It can take actions. The following require an explicit "yes" at the moment the action is about to happen — not generic permission granted at session start.

Developer

Modifying files to be committed; running scripts outside a sandbox; making network calls; touching schemas or migrations; modifying shared configuration files.

Data & Analytics

Running queries on production or staging; publishing to shared dashboards; modifying pipeline definitions; changing schema definitions or table structures.

PM / Product

Generating communications going externally; updating shared project records; producing documents for clients or external stakeholders.

Ops & Support

Any command touching production infrastructure; modifications to shared configs; scripts interacting with live monitoring, alerting, or paging systems.



The rule: If Claude Code is about to do something irreversible, or something others will see — it needs your explicit approval at that moment.

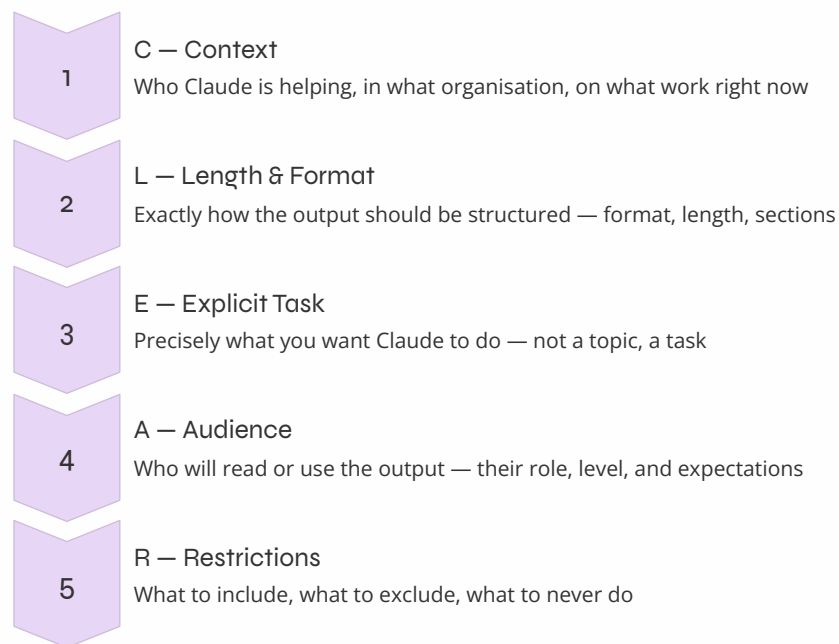
Question 3: What Must Always Be Reviewed?

Everything. Code before it is merged. Reports before they are sent. Requirements before sign-off. Runbooks before following in an incident. Emails before sending. SQL before it runs on real data. Test cases before entering the regression suite.

Claude Code produces drafts. You produce the final. The tool handles generation and structure. You bring judgement, accountability, and the context the tool cannot see.

The CLEAR Framework — The Anatomy of a Good Prompt

One universal skill determines the quality of everything Claude produces: how you ask. Most people begin with prompts like *"Write a status report"* or *"Review this code."* These are not prompts. They are topics. A topic tells Claude what domain you are in. A **prompt** tells Claude what to produce, for whom, in what format, under what constraints.



☐ The CLEAR framework applies equally in Claude.ai, Projects, Co-work, and Claude Code. It is the **universal prompting standard** across all four products. Every prompt in the role-specific libraries in Part 7 is already CLEAR-structured.

Three Rules That Apply to Every Role, Every Day



Rule 1: Claude Drafts. You Decide.

Never auto-send, auto-commit, auto-publish, or auto-deploy anything Claude produces. You are accountable for the output. Accountability requires review — every time, without exception.



Rule 2: Wrong Output Is a Diagnostic

When Claude gets something wrong, do not re-ask the same question. Ask: *"What context is missing that caused this?"* That missing context goes into your CLAUDE.md or memory.md. Every correction makes the next output better. The quality compounds — but only if you use failures as feedback.



Rule 3: Specificity Is the Work

Precise, contextual instructions are the primary leverage point. A prompt library built over one month saves hours every week. A CLAUDE.md built thoughtfully in an afternoon shapes every session for years.



Part 4

The Context Files

What They Are, Why They Exist, How They Work Together

Claude Code operates through four types of context files. Understanding what each does — and what it is *not* for — is the foundation of everything that follows. These files are what transform Claude Code from a capable generic tool into one that knows your work.

The Four Context Files Explained

1

CLAUDE.md — Your Operating Manual

Read automatically at session start. Contains your **Rules** (what data never enters, what requires approval, what must be reviewed), **Context** (who you are, what you are working on, what tools you use), and **Preferences** (tone, format, how Claude handles uncertainty). The most important thing you build today.

2

memory.md — Persistent Project Context

CLAUDE.md tells Claude how to work with you. memory.md tells Claude what is true about your project and domain. Decisions made, known issues, team context, data quality problems — the specifics of *your* work rather than a generic version of it. A living document that grows richer over time.

3

SKILL.md — Reusable Instruction Set

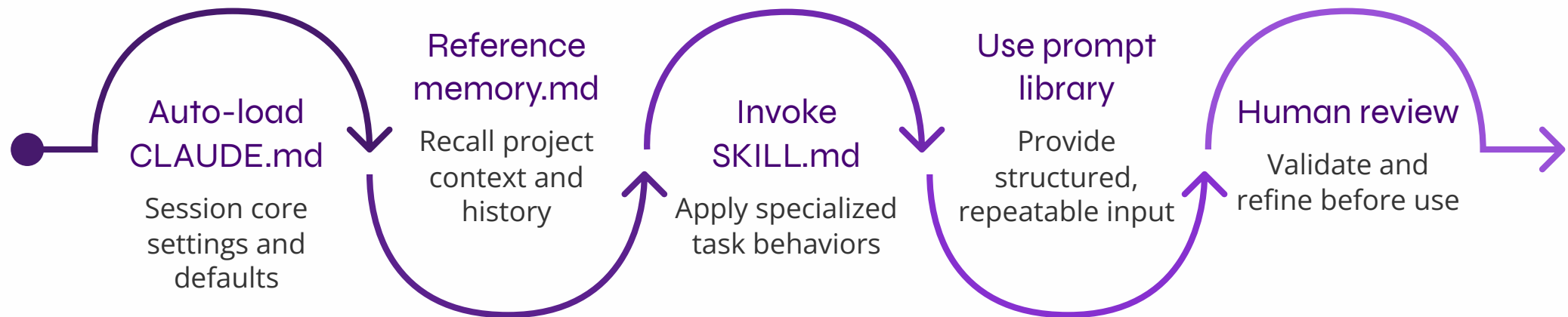
A detailed, reusable instruction set for one specific repeatable task. Includes exact structure, input requirements, quality checklist, and failure modes to avoid. Built once. Consistent, fast, and quality-controlled every time it is invoked. Portable and shareable with teammates.

4

Prompt Library — Starting Point for Every Task

A curated collection of prompts ready to use for your most common tasks. Eliminates starting from a blank page. Encodes best practices — every prompt is already structured to produce the right output for the right audience in the right format. Grows over time.

How the Files Work Together in Every Session



The power of this system is cumulative. Each layer adds specificity. Each session builds on the last. The output quality you experience in month three is fundamentally different from day one — because the context behind every request has grown richer and more precise.

Part 6

The Four Phases — How Practices Mature Over Time

Phase progression is not mandatory. It is descriptive. You advance as your context deepens and confidence grows. **Today covers Phase 1 and Phase 2** — the foundation everything else builds on.

Phase 1 — Foundation

Rules. Safety. First prompts. Artifacts: CLAUDE.md, Prompt Library, Safety Checklist, Reference Card.

Phase 2 — Daily Use

Personal context. Habits. Reusable skills. Artifacts: memory.md, SKILL.md (per role), CLAUDE.md deepened, Workflow Map.

Phase 3 — Integration

Your tools. Your team. Shared context. Artifacts: MCP config, Shared CLAUDE.md, Eval Checklist.

Phase 4 — Automation

Agents. Hooks. End-to-end workflows. Artifacts: Agent instruction file, settings.json (hooks), Team Playbook.

Part 7 — Role-Specific Tracks

Find Your Role. Work Through It Completely.

Each role track includes specific safety rules, a CLAUDE.md context template, memory.md guidance, your first SKILL.md, and a ready-to-use prompt library. Work through your role before reading any other.

Role 1 Developer	Role 2 Data & Analytics
Role 3 QA & Testing	Role 4 Project / Product Manager
Role 5 Business Analyst	Role 6 Ops & Support



Role 1

Developer

Claude Code is not autocomplete and it is not a Stack Overflow replacement. Used well, it is a collaborator that works at your code level — understanding your conventions, your architecture, your constraints — and produces output that fits your actual codebase. The central risk in your role is **incorrect code reaching production**. That risk is managed through your rules, your review discipline, and the quality controls you build into your skills.

Developer: Safety Rules + Context

Data — Never in a Prompt

- Production credentials, API keys, secrets, tokens of any kind
- Database connection strings or authentication details
- Live customer records, user tables, or PII from any environment
- Internal IP addresses, network topology, security configurations
- Data subject to GDPR, HIPAA, or applicable data protection regulation

Actions Requiring Explicit Approval

- Modifying any file to be committed to a shared branch
- Running any script or migration outside an isolated sandbox
- Making any external network call
- Touching database schemas or seed data

Your CLAUDE.md Context Section

Write your own version of each section below — use the example as a guide, not a template to copy.

- **Who I am:** Your role, team, stack, domain. Write as if briefing someone on their first day.
- **What I am working on right now:** Current project, phase, main risk.
- **My tools:** VS Code, GitHub, Jira, Docker, cloud provider, monitoring tool.
- **Conventions:** Language, framework, naming rules, patterns in active use, anti-patterns to avoid — e.g. no any types, no console.log in production, camelCase functions, PascalCase components.

Developer: First SKILL.md — Code Review

This skill is invoked when reviewing any PR before it is merged to main. Adapt the quality checklist to your actual codebase standards.

Input Required

- The code diff or files changed
- The Jira ticket and acceptance criteria
- The PR description

Output Produced

Three sections: **Critical issues** (must fix before merge, with explanation and specific fix), **Minor issues** (should fix), **Suggestions** (optional improvements). Explains what is wrong and why it matters — does not rewrite the code.

Quality Checklist

- Every critical issue has a clear explanation and specific recommended fix
- No issues contradict stated conventions in CLAUDE.md
- Review is actionable — author knows exactly what to do
- Security issues are always marked critical, never minor

Anti-patterns

- Do not rewrite code — explain the issue only
- Do not flag style preferences as critical issues
- Do not approve code with new functionality and no tests
- Do not invent method names — if unsure, flag for human verification

Developer: Prompt Library

Save these in a file called `prompt-library.md`. Fill in the bracketed fields for your context. Every prompt is already CLEAR-structured.

→ **Code Review**

Review for: security vulnerabilities, performance issues, convention adherence, test coverage gaps. Format: Critical issues | Minor issues | Suggestions. Do not rewrite — explain each issue and why it matters.

→ **Unit Test Generation**

Generate tests using [Jest/pytest/JUnit]. Structure: Arrange, Act, Assert. Cover happy path, edge cases, failure modes. Name: `should_[behaviour]_when_[condition]`. Test behaviour, not implementation.

→ **PR Description**

Format: title (imperative verb, under 72 chars), summary (what and why), changes made, testing done, breaking changes. Factual. No marketing language. No "this PR aims to."

→ **Debug Assistance**

Do not rewrite the function. Explain: what is wrong, why it is happening, what the specific fix is. If multiple possible causes, list in order of likelihood with how to confirm each.

→ **Commit Message**

Write in Conventional Commits format. Type(scope): description under 72 characters. Add body paragraph if the change needs explanation.

Role 2

Data & Analytics

Data work has a specific failure mode that makes it different from every other role: **hallucinated specificity**. Claude can produce SQL that looks completely correct and runs without error — but references the wrong table, the wrong column, or uses a metric definition that differs from your team's actual definition by three lines of logic. The output looks right. It is wrong. This is why **memory.md** is **foundational** for your role.



Data & Analytics: Safety + memory.md Priorities

Data — Never in a Prompt

- Raw production tables with customer identifiers or PII fields
- Unreleased financial metrics, forecasts, or performance data
- Model training datasets containing personal data
- Query results from production that include PII — even a sample
- Data that could identify individuals when combined with other fields

Your memory.md — Most Critical Entries

- **Schema definitions:** Critical tables, key columns, known quirks — e.g. event_timestamp is UTC but created_at is local server time. Silent errors live here.
- **Metric definitions:** The exact SQL, exact filters, exact table and field behind every KPI. Not "revenue" — the precise calculation.
- **Known data quality issues:** NULL problems, duplicate issues from migrations, anything Claude will run directly into if it does not know.
- **Pipeline dependencies:** What feeds what. What breaks downstream when a pipeline fails.

Data & Analytics: Prompt Library

SQL Generation Write a SQL query for [platform]. Paste schema and metric definitions from memory.md. Conventions: snake_case, CTEs over subqueries, WHERE clause on large tables, comment non-obvious logic, no SELECT *. Flag any schema assumptions.	Data Quality Check Check for: nulls in non-nullable fields, out-of-range values, duplicates, referential integrity, format inconsistencies. Format: Issue Affected column Severity Recommended action. Paste known issues from memory.md.	EDA Summary Write a summary for a non-technical business stakeholder. Format: 3-sentence executive summary, key findings (bullet, lead with the number), anomalies, recommended next steps. No SQL, no field names, no jargon.	Metric Definition Write a formal metric definition for the data catalog. Include: business definition (plain language), calculation logic, data source, refresh frequency, known caveats, owner. No SQL in the business definition section.
Pipeline Documentation Write technical documentation for a data pipeline. Include: purpose, sources, transformation logic, output schema, dependencies, failure modes and recovery steps, owner. Audience: a data engineer who has never seen this pipeline.			



Role 3

QA & Testing

Testing has a paradox at its centre: tests that pass are not necessarily tests that catch bugs. A Claude-generated test suite that is syntactically correct and runs green can still miss the failure modes that matter. Your role with Claude Code is not to generate tests faster. It is to generate tests that are **actually useful** — that cover the right scenarios, catch real bugs, and that a developer reading them two years from now can understand. That requires your context.

QA: First SKILL.md — Test Case Generation

Input Required

- Feature description and acceptance criteria
- Persona(s) or user types involved
- Known constraints, dependencies, or integration points
- Regression history from memory.md

Output Produced

Format: ID | Description | Preconditions | Steps | Expected result | Priority (P1/P2/P3). Covers happy path, edge cases, negative cases, and concurrency where applicable. **Test cases generated first — not code.** Code comes after cases are reviewed.

Quality Checklist

- Happy path covered with at least one positive test
- Every boundary value has a test (min, max, min-1, max+1)
- At least one negative test per input type
- Past regression failures from memory.md are explicitly covered
- No test requires another test to have run first

Anti-patterns

- Do not generate automation code until test cases are reviewed
- Do not write tests that test implementation details
- Do not mark a P1 as P2 to improve coverage numbers

QA: Prompt Library

→ Test Case Generation

Cover: happy path, edge cases, negative cases, boundary values, concurrency if applicable. Format: ID | Description | Preconditions | Steps | Expected result | Priority. Generate test cases only — not automation code. Paste framework context and regression history.

→ Regression Impact Assessment

Given the changes in this release, identify which test suites are most at risk. For each: why it is at risk, what testing is needed, priority. Reference regression history from memory.md. Format: Affected area | Reason | Test type | Priority.

→ Bug Report

Format: Summary (one line, specific), Environment, Steps to reproduce (numbered, exact), Expected behaviour, Actual behaviour, Severity with justification. Precise language. Only observed behaviour — no speculation about cause.

→ Automation Script

Write an automated test using [Cypress/Appium/Jest]. Test user-observable behaviour, not implementation. Include setup, teardown, and a comment block. One clear assertion focus per test. Paste conventions from SKILL.md.

Role 4

Project / Product Manager

The primary risk in your role is not technical. It is **reputational and relational**. A status report with AI-generated text that hasn't been reviewed can contain wrong dates, misrepresented risks, or inaccurate RAG status. A stakeholder email sent without review can strike the wrong tone, omit a critical nuance, or commit to something that wasn't agreed. The rule for your role is simple and absolute: **nothing that leaves your hands goes out without your eyes on it.**



PM: First SKILL.md — Weekly Status Report

Input Required

- Raw updates from each workstream lead
- Decisions made this week
- New risks identified
- Decisions needed from the client
- Current risk log from memory.md

Output: One-Page Status Report

- Headline summary: 2 sentences — what happened, what matters
- RAG status table: workstream | status | one-line reason
- Top 3 risks: description | likelihood | impact | mitigation
- Decisions needed: specific questions requiring client response

Quality Checklist

- RAG status is accurate — no green-washing of amber or red situations
- Bad news is stated clearly and paired with the mitigation plan
- No jargon the client would not understand
- Under one page
- Decisions needed are specific — not "please advise" but "do you approve X so we can proceed with Y by [date]?"
- Every risk has a mitigation, not just an observation

Anti-patterns

- Do not soften bad news — state it with the mitigation
- Do not use filler: "significant progress was made," "we are working hard to resolve"
- Do not exceed one page

PM: Prompt Library

Weekly Status Report

Format: headline summary (2 sentences), RAG status table per workstream, top 3 risks with mitigation, decisions needed. Tone: direct, factual, no jargon. Maximum 1 page. Do not soften bad news — state it with the mitigation plan.

Meeting Summary

Format: decisions made (numbered), action items (owner | action | due date), open questions, 3-sentence meeting summary. Flag anything unclear as [UNCLEAR] — do not guess or infer. Notes: [paste].

Stakeholder Email

Write for [name/role] about [topic]. Tone: [professional/direct/diplomatic]. Must include: [specific points]. Must not include: [what to leave out]. Under 200 words unless complexity requires more. Paste stakeholder context from memory.md.

Risk Log Update

Update existing risks if status, likelihood, or mitigation has changed. Add new risks identified this week. Format: ID | Description | Likelihood | Impact | Owner | Mitigation | Status. Do not remove risks without explicit instruction.

Roadmap Narrative

Explain the sequencing and trade-offs: why things are in this order, what was deprioritised and why. Audience: [describe]. Confident and direct. No hedging. No filler. 3–4 paragraphs. Paste stakeholder context from memory.md.

Role 5

Business Analyst

The central challenge of your work is that the raw material is almost always **ambiguous**. Stakeholders say one thing and mean another. Requirements contain contradictions that only become visible when placed next to each other. Claude Code is a tool for structuring ambiguity — but it cannot resolve it. Claude can take raw workshop notes and produce structured user stories. **You must catch the ones where structure created false clarity** — where a story looks clean but contains an assumption no stakeholder actually agreed to.



BA: First SKILL.md — User Story Writing

Output Per Story

- Title
- Story statement: As a [persona], I want [goal], so that [benefit]
- Acceptance criteria in Gherkin: Given / When / Then (minimum 2 scenarios)
- Priority: Must have / Should have / Could have
- Assumptions (explicit — not embedded in the story)
- Dependencies or blockers

Quality Checklist

- Every story is independently testable
- Acceptance criteria use concrete, measurable language only
- Every story fits in one sprint — oversized stories flagged
- No invented requirements — only what stakeholders actually said
- Domain vocabulary from memory.md used consistently

Anti-patterns

- **Do not invent requirements** — only structure what stakeholders said. This is the most important rule in this skill.
- Do not write technical implementation detail into stories
- Do not combine multiple user goals into one story
- Do not use subjective language in acceptance criteria ("easy to use," "fast," "intuitive")

memory.md — Most Critical Entries for BA

- **Domain vocabulary:** Every term with a specific meaning different from general usage. Claude will use the general definition if you do not provide the domain one.
- **Business rules:** The logic governing what is allowed, required, forbidden.
- **Stakeholder map:** Who owns what, who has sign-off authority, who raises specific types of objection.

BA: Prompt Library

→ User Story Writing

Format: story statement, acceptance criteria (Gherkin: Given/When/Then — min 2 scenarios), priority, assumptions, dependencies. Stories must be independently testable and sprint-sized. Flag requirements too vague to write a testable story for — do not invent clarity. Paste domain vocabulary from memory.md.

→ Process Documentation

Structure: purpose, trigger, inputs, roles, step-by-step procedure (numbered), decision points (explicit), outputs, exceptions, related processes. Numbered steps. Plain language. Active voice. Paste domain vocabulary from memory.md.

→ Gap Analysis

Categorise by: people, process, technology, data, governance. For each gap: description, impact if not addressed, suggested action. Format: Category | Gap | Impact | Action | Priority. Paste current state and future state.

→ Stakeholder Interview Guide

Create an interview guide for a workshop on [topic]. Include: objectives, opening questions, deep-dive questions, challenge questions (to surface unstated assumptions), closing questions. Open-ended questions only. Paste stakeholder context from memory.md.

Role 6

Ops & Support

Your role has the sharpest edge of any on this list. At 2am during a P1 incident, under pressure, with an alert you have not seen before — **the temptation to have Claude Code "figure it out" is real. This is the moment where the boundaries matter most.** Claude Code does not know your production environment. It can produce a runbook that looks correct and is missing the one step that matters. The value of Claude Code in your role is in the *non-incident work*: building runbooks before the incident, drafting postmortems after, documenting failure modes while you still remember them.



Ops: Safety Rules — The Strictest on the List

Data — Never in a Prompt

- Production credentials, passwords, API keys, tokens
- Customer data captured in logs — even partial records
- Security audit results, vulnerability scans, penetration test findings
- Network topology, internal IP ranges, or infrastructure configurations

Actions — Always, No Exceptions

- Any command touching production infrastructure
- Any modification to shared configuration files
- Any script interacting with live monitoring, alerting, or paging systems
- Any service restart, scale event, or infrastructure change
- Any change to access controls or security configurations

Always Review Before Use

- Every runbook before it is followed in a real incident
- Every postmortem before it is shared
- Every alert triage before action is taken
- Every automation script before deployment

Ops: Prompt Library

Incident Summary

Format: ID and title, severity, start and resolution time, impact (quantified), timeline (numbered, timestamped), immediate cause, contributing factors, actions taken, current status. Blameless. Factual. Mark any unconfirmed items as [unconfirmed]. Always paste system context from memory.md.

Runbook

Structure: trigger condition, pre-conditions to verify, step-by-step procedure (numbered), verification steps, rollback procedure, escalation path. Write for someone on-call at 2am with no additional context. Highlight destructive or irreversible steps. **Every runbook must include a rollback procedure — if there is none, flag it explicitly.**

Postmortem

Format: incident summary, impact, timeline, root cause analysis (5 Whys), contributing factors, what went well, what went poorly, action items (description | owner | due date | success criteria). Focus on systems and processes — never individuals. Must include at least one preventive action and one detection improvement.

Alert Triage

List the top 3 most likely causes in order of probability. For each: what it is, why it is likely given the context, what to check first to confirm or rule it out. **Do not recommend production actions — diagnostic steps only.**

Toil Identification

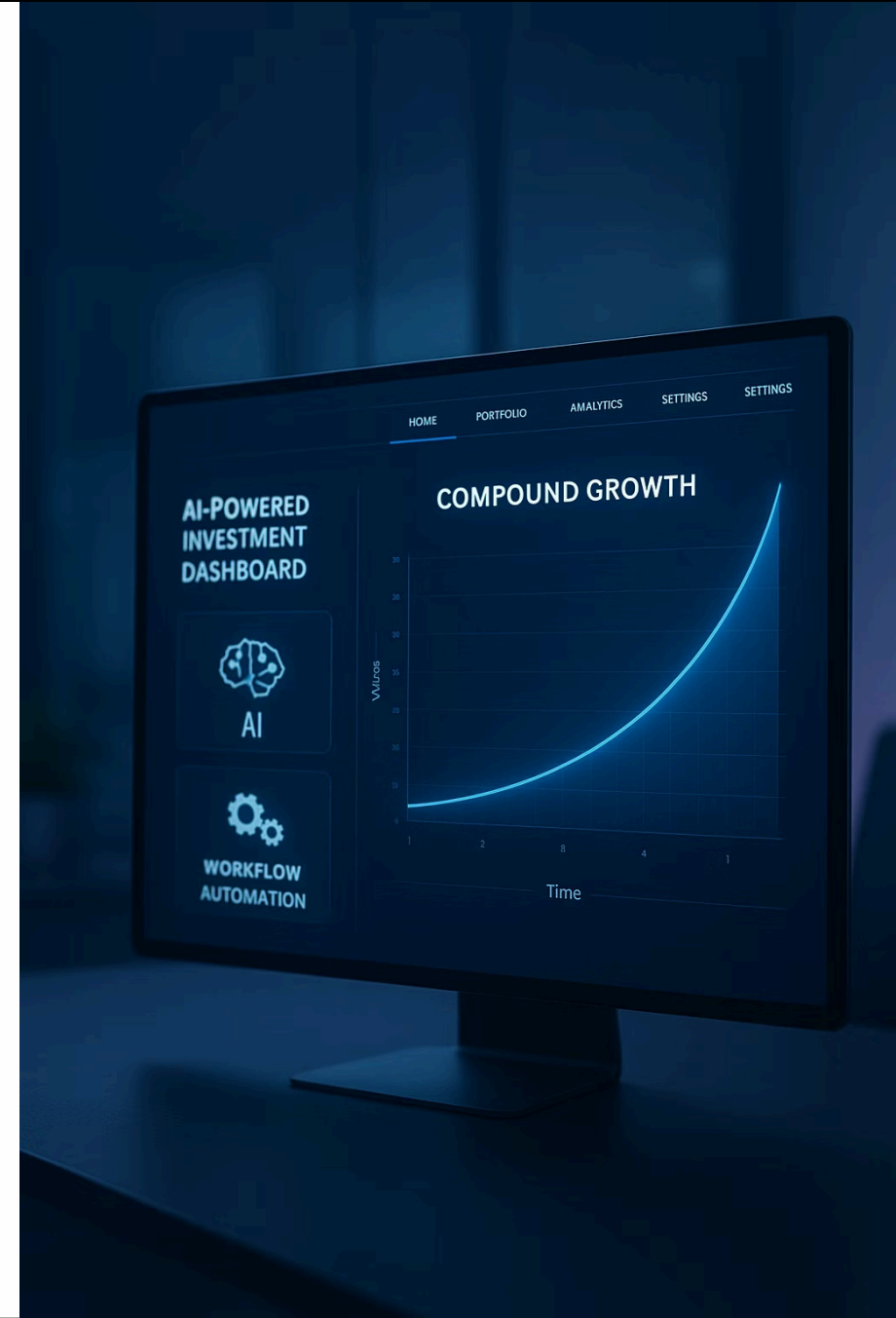
Evaluate each task by: frequency, time cost per occurrence, error rate, repeatability. For each candidate: description, estimated time saved, automation complexity (low/medium/high), suggested approach. Prioritised list — highest ROI first.

Part 8

The Compound Effect

Why This Gets Better Over Time

There is a common misconception about AI tools: that the value is fixed. That what you get today is what you will always get. Claude Code is the opposite of that. The quality of every output is determined by the quality of the context behind it. **And context builds.**



The Compounding Timeline



📌 Every time Claude produces wrong output and you update your artifacts — you have made a permanent improvement. Not just for today. For every session that follows.

Part 9

The Full Artifact Map

What Every Role Builds, Across All Four Phases

Artifact	Phase	Dev	Data	QA	PM	BA	Ops
Safety Checklist	1	●	●	●	●	●	●
Reference Card	1	●	●	●	●	●	●
Prompt Library	1	●	●	●	●	●	●
CLAUDE.md	2	●	●	●	●	●	●
memory.md	2	●	●	●	●	●	●
SKILL.md	2	●	●	●	●	●	●
Workflow Map	2	●	●	●	●	●	●
MCP Config	3	●	●	●	●	●	●
Shared CLAUDE.md	3	●	●	●	●	●	●
Eval Checklist	3	●	●	●	●	●	●
Agent Instruction File	4	●	●	●	—	—	●
settings.json (Hooks)	4	●	—	●	—	—	●
Team Playbook	4	●	●	●	●	●	●

📌 Phase 3 and Phase 4 deep-dives are covered in the next session. Today: Phase 1 and Phase 2.

Part 10

Three Supporting Artifacts That Complete Your Foundation

Your CLAUDE.md, memory.md, SKILL.md, and prompt library are your primary tools. Three supporting artifacts round out your Phase 1 and 2 foundation. They take minutes to create and prevent the most common mistakes.



Safety Checklist

A one-page personal checklist you run before any Claude session that involves real work data. Turns the safety rules from Part 3 into a habit, not a memory exercise. Checks: before you paste anything in, before Claude takes an action, before you use any output.



Reference Card

A one-page quick reference you keep open during Claude sessions. Contains the CLEAR framework and your most-used prompt structures — so you never have to remember them, just glance. Print it or keep it as a pinned tab.



Workflow Map

A one-page diagram of where Claude fits into your actual daily work. Not aspirational — descriptive. Where do you currently spend time on repeatable, structured tasks? Those are the Claude insertion points. Includes a "never" column for where Claude does not belong.

The Workflow Map — What to Document

A Workflow Map does not need to be a formal diagram. It is a personal tool for identifying where Claude creates the most leverage in your daily work.

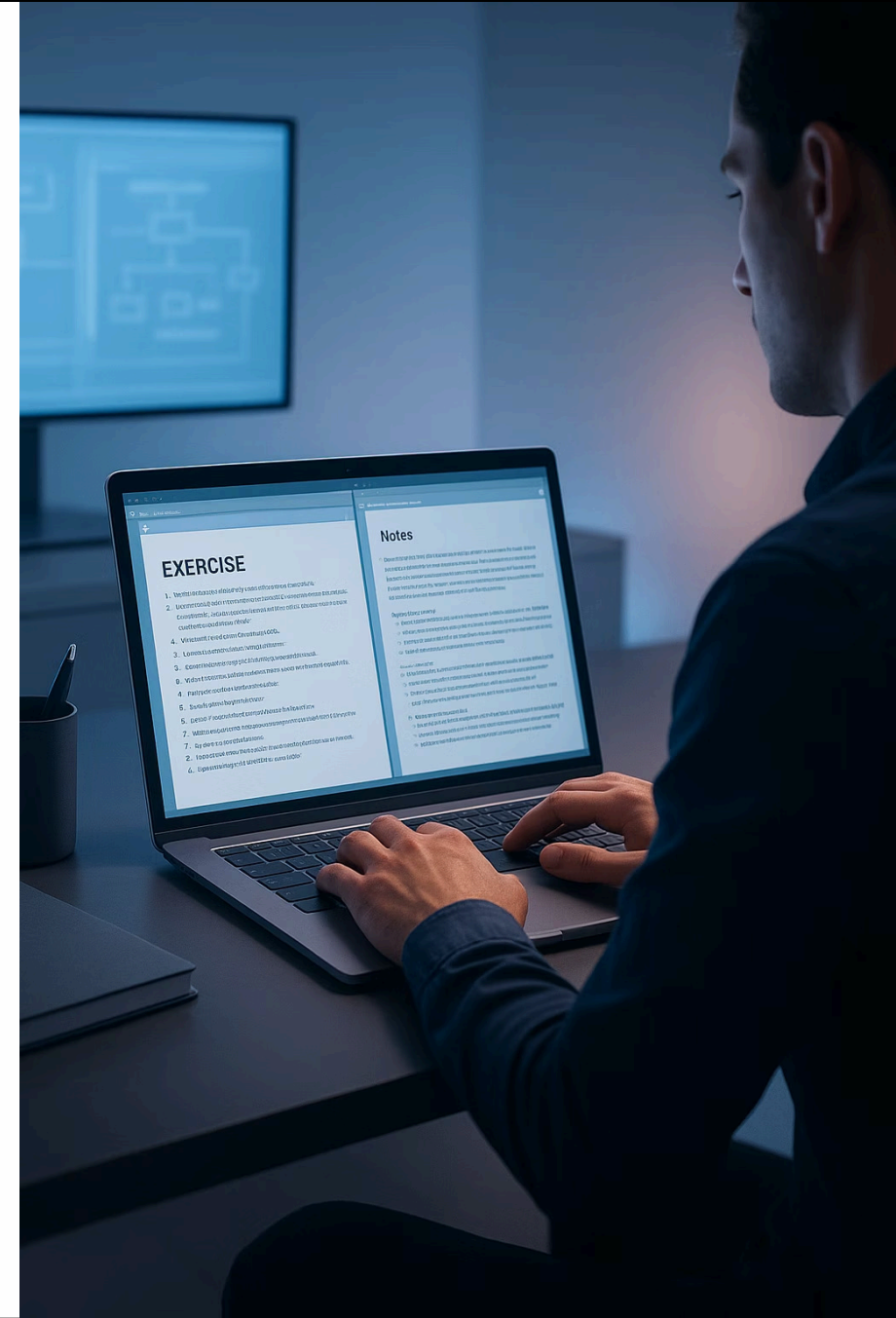
Task	Which product	Which artifact/prompt	Time saved
Weekly status report	Claude Code	SKILL: Weekly Status Report	~45 min
Code review	Claude Code	SKILL: Code Review	~30 min
Meeting summary	Claude.ai	Meeting Summary prompt	~20 min
SQL query draft	Claude Code	SKILL: SQL Generation	~25 min
[Your task here]			

📌 **The "never" column matters.** Knowing where Claude does not belong is as important as knowing where it does. Document it explicitly — it reinforces your boundaries and builds trust in the system.

Part 11

Capstone: Your First End-to-End Workflow

You have built the artifacts. Now use them. Each role has one capstone task — a real piece of work that uses CLAUDE.md, memory.md, a SKILL.md, and at least one prompt from your library in sequence. **This is not an exercise. Do it with actual work from your job.**



Capstone Tasks by Role

Developer

Take a real piece of code you wrote or reviewed recently. Invoke SKILL: Code Review, review the output, then generate unit tests for the same code. Update your SKILL.md with one new anti-pattern based on what you observed.

Data & Analytics

Take a real metric or report your team produces regularly. Use SKILL: SQL Generation with actual schema from memory.md. Review the query for correctness, then turn the expected output into a stakeholder summary using your EDA Summary prompt.

QA & Testing

Take a real feature currently in development or recently shipped. Use SKILL: Test Case Generation with actual acceptance criteria. Review against regression history. Use your Bug Report prompt to write up a real recent bug.

PM / Product Manager

Take last week's status update — the raw notes before you formatted it. Use SKILL: Weekly Status Report. Review: is RAG status accurate? Is bad news stated clearly? Then use the Risk Log Update prompt with this week's actual developments.

Business Analyst

Take notes from a real requirements workshop. Use SKILL: User Story Writing with the actual notes. Review for invented requirements and subjective criteria. Then use Acceptance Criteria Review on the generated stories.

Ops & Support

Take notes from a real incident — recent or historical. Use SKILL: Incident Summary with the actual timeline and notes. Review for blamelessness and factual accuracy. Then use the Postmortem prompt to generate the full postmortem.

After the Capstone — One Update Rule

Based on what you just observed in the capstone, make exactly one update to your artifacts before the session ends:

Something Claude got wrong

Update CLAUDE.md or memory.md with the missing context that caused the error.



Something the skill missed

Update the relevant SKILL.md — add it to the quality checklist or anti-patterns section.



A prompt that needed significant editing

Improve that prompt in your library so the next time you run it, you start from a better place.



One update. Not a rewrite. The compound effect starts with a single correction. The discipline of making that one update — every session, every time — is what separates a context file that plateaus from one that keeps getting better.



Part 12

Closing: What You Built. What Comes Next. Why It Matters.

You have built version 1.0 of something that will be working for you long after this session ends. It is not perfect. It will produce wrong output. That is not a failure of the system — it is the system working as designed. Every wrong output tells you something: what context was missing, what rule was not specific enough, what assumption Claude made that you did not catch. Update your CLAUDE.md. Update your memory.md. **The quality compounds.**

The Path Forward

Today Safety rules defined. CLAUDE.md, memory.md, SKILL.md v1. Prompt library started. The foundation is in place.	This Week Use your prompt library in real work every day. Every wrong output: update an artifact. Every new prompt that works: add it to the library.	Next Session Phase 2 deepened. Phase 3 begins: tools connected, team context shared. MCP configuration introduced.	After That Phase 3 complete: shared CLAUDE.md, eval checklists, MCP connected. Everyone on the team working from the same context.
Phase 4 Agents. Hooks. Team playbook. The repeatable work runs itself. You focus entirely on the work that requires you.			

One Question to Take With You

Before the next session, notice the moments in your day where you think: **"This takes too long. I do this same thing every week. The structure never changes."** Those are your next SKILL.md files.

4

Context File Types

CLAUDE.md, memory.md, SKILL.md, Prompt Library
— built today, compounding every day after.

4

Products in the Ecosystem

Claude.ai, Projects, Co-work, Claude Code —
different tools for different jobs.

6

Role Tracks Covered

Developer, Data, QA, PM, BA, Ops — each with
specific safety rules, skills, and prompt libraries.

4

Phases of Maturity

Foundation → Daily Use → Integration →
Automation. Today: Phases 1 and 2.

Key Takeaways



Build the foundation, not the feature list

The underlying model — CLAUDE.md, memory.md, SKILL.md — is stable. New features extend it. Master the foundation and every new capability is an extension of something you already understand.



Safety rules are preconditions, not optional extras

Answer the three safety questions before any real work. What data never enters a prompt. What requires explicit approval. What must always be reviewed. These are not suggestions — they are the conditions under which the tool can be trusted.



Context is your primary leverage point

Claude is only as useful as the context you give it. The work of building that context is a one-time investment that compounds every day after. Specificity is the work.



Wrong output is a system feedback loop

Every wrong output is a signal about missing context. Every correction to your artifacts is a permanent improvement. The compound effect starts with a single update after a single mistake.



Thank You

A month from now your context files will look nothing like what you built today. They will be specific to your actual work in a way no template could produce — because the context is yours. You built it.

CLAUDE CODE TRAINING

PHASE 1 + 2 COMPLETE